

ic-memory

Stable-Memory Allocation Governance for Upgrade-Safe Internet Computer Canisters

Adam Powell

May 31, 2026

Abstract

Internet Computer canisters often store durable state through **ic-stable-structures**. Multiple stable stores are commonly multiplexed through a **MemoryManager**, where each logical store is assigned a physical stable-memory slot. Across upgrades, an accidental reassignment can make a canister open one store’s bytes as another store’s data. **ic-memory** addresses this class of failure by maintaining a durable allocation ledger that binds stable logical keys to physical **MemoryManager** IDs and rejects layout drift before stable-memory handles are opened. This paper presents the problem, protocol, core invariants, and a compact Lean mechanization of the allocation-safety argument.

1 Problem Statement

Stable memory is durable across Internet Computer canister upgrades. That durability is useful only when future binaries interpret the same durable bytes under the same logical identity. Consider a canister version that ships with

```
app.users.v1 ↦ 100,  
app.orders.v1 ↦ 101.
```

If a later binary accidentally ships with the assignments swapped, the program may compile and install while opening orders data as users data. The failure is not a memory-access violation; it is an allocation-ABI violation. **ic-memory** treats the mapping

stable key $\longrightarrow$ physical allocation slot

as persistent protocol state. A future binary may reopen the same key on the same slot, or it may introduce a new key. It may not move an existing key to a new slot, and it may not reuse an existing slot for another active key.

2 Scope

The crate is allocation-governance infrastructure. It does not implement stable collections, prove application schema compatibility, authorize controllers, or perform data migrations. Its narrower responsibility is to validate declarations against historical allocation facts before opening memory.

The native substrate is the **MemoryManager** from **ic-stable-structures**. Its usable ID domain is

$$S = \{0, 1, \dots, 254\},$$

with ID 255 reserved by **ic-stable-structures** as the unallocated sentinel. In the default runtime, **ic-memory** reserves IDs 0..=9 for governance and uses ID 0 for the allocation ledger. Lower-level bootstrap owners that bypass the default runtime must enforce equivalent policy themselves.

3 Substrate Correspondence

The allocation slot model is intentionally aligned with the current `ic-stable-structures` `MemoryManager` implementation. The upstream manager defines `MemoryId` as a one-byte value, uses 255 as the unallocated-bucket marker, and stores each bucket owner as one byte in the manager metadata. Therefore `ic-memory`'s current slot descriptor is not an arbitrary namespace: it is the exact durable ID domain that the underlying manager can distinguish. The same implementation reserves the first stable-memory page for manager metadata. Its V1 header stores the magic value `MGR`, a layout version, the allocated-bucket count, the bucket size, 32 reserved bytes, and one page-count entry for each managed memory $0, \dots, 254$. The bucket-owner table follows the header. With the default bucket size of 128 Wasm pages and 32,768 managed buckets, the manager can address 256 GiB of bucket-backed stable memory through this layout. `ic-memory`'s default ledger anchor also relies on `ic-stable-structures::Cell`. The relevant Cell V1 envelope begins with magic `SCL`, a one-byte layout version, and a four-byte value length before the encoded value. `ic-memory` preflights that envelope before opening the ledger Cell so corrupt ledger storage is classified as a bootstrap error instead of escaping as a decode panic.

4 Protocol Overview

Each binary contributes a declaration snapshot D , containing pairs

$$(k, s) \in K \times S$$

where k is a stable key such as `app.users.v1`, and s is a physical `MemoryManager` ID. The runtime then recovers the committed allocation ledger, collects current declarations, validates them against history and policy, commits the next generation, and publishes a capability that can open validated slots.

The default runtime performs that sequence before publishing its open authority. The lower-level Rust APIs expose the pieces separately for framework owners, so manual integrations must preserve the same order. The ordering is the central safety boundary:

recover $\rightarrow$ validate
 $\rightarrow$ commit $\rightarrow$ open

Opening stable-memory handles before this boundary defeats the protocol.

5 State Model

Let K be the set of stable keys and S the set of usable physical slots. A ledger is a finite sequence of records

$$L = [r_1, \dots, r_n].$$

Each record has

$$r = (k, s, \text{state}, \text{first}, \text{last}, \text{retiredAt})$$

where $k \in K$, $s \in S$, and

$$\text{state} \in \{\text{Reserved}, \text{Active}, \text{Retired}\}.$$

Definition 1 (Active binding). *A stable key k is active at slot s in ledger L , written $\text{ActiveAt}(L, k, s)$, when*

$$\exists r \in L. r.\text{key} = k \wedge r.\text{slot} = s \wedge r.\text{state} = \text{Active}.$$

Definition 2 (Retired binding). *A stable key k is retired at slot s , written $\text{RetiredAt}(L, k, s)$, when*

$$\exists r \in L. r.\text{key} = k \wedge r.\text{slot} = s \wedge r.\text{state} = \text{Retired}.$$

6 Allocation Invariants

Invariant 1 (No stable-key movement). *For all records $r_1, r_2 \in L$,*

$$r_1.\text{key} = r_2.\text{key} \Rightarrow r_1.\text{slot} = r_2.\text{slot}.$$

Once a stable key is assigned to a slot, that key cannot later name another slot.

Invariant 2 (No physical-slot reuse). *For all records $r_1, r_2 \in L$,*

$$r_1.\text{slot} = r_2.\text{slot} \Rightarrow r_1.\text{key} = r_2.\text{key}.$$

Once a physical slot is assigned to a stable key, the slot cannot be reused for a different stable key.

Invariant 3 (Retirement is a tombstone). *For all $k \in K$ and $s \in S$,*

$$\text{RetiredAt}(L, k, s) \Rightarrow \neg \text{ActiveAt}(L, k, s).$$

Retirement is not a free-list operation. It preserves historical ABI facts for rollback safety and diagnostics.

Invariant 4 (Post-commit open authority). *In the abstract model, a memory handle is opened through a post-commit authority tied to a committed ledger. If that authority permits opening k at s , the committed ledger contains an active record for k at s :*

$$\text{MayOpen}(A, k, s) \Rightarrow \text{ActiveAt}(L, k, s).$$

For the Rust default runtime, this corresponds to publishing validated allocations only after the staged ledger generation has been committed. Manual integrations carry this as an ordering obligation.

Invariant 5 (Generation monotonicity). *Successful staging advances the committed ledger generation by exactly one in the abstract model:*

$$g' = g + 1 \quad \text{and therefore} \quad g < g'.$$

The Rust implementation refines this with explicit checks for stale validated generations, declaration-count bounds, and `u64` overflow.

7 Proof Sketch

Theorem 1 (Stable key has a unique slot). *Assume a ledger L satisfies no stable-key movement. If records r_1, r_2 are both in L and $r_1.\text{key} = r_2.\text{key}$, then $r_1.\text{slot} = r_2.\text{slot}$.*

Proof. This is direct application of the no stable-key movement invariant. The interesting implementation obligation is therefore not the theorem itself, but ensuring every recovery, validation, reservation, staging, and commit path preserves the invariant before producing authority. $\square$

Theorem 2 (Physical slot has a unique stable key). *Assume a ledger L satisfies no physical-slot reuse. If records r_1, r_2 are both in L and $r_1.\text{slot} = r_2.\text{slot}$, then $r_1.\text{key} = r_2.\text{key}$.*

Proof. This is direct application of the no physical-slot reuse invariant. The result prevents the swapped-ID failure: two distinct keys cannot both own the same stable-memory slot. $\square$

Theorem 3 (Open authority is ledger-backed). *Let A be a post-commit open authority tied to committed ledger L . If A permits opening stable key k at slot s , then L contains an active allocation record for k at s .*

Proof. By model construction, a post-commit open authority contains only declarations whose matching allocation records are active in the committed ledger. Opening resolves a stable key to one of those declarations, so the corresponding active ledger record exists. $\square$

Theorem 4 (Generation advancement is monotone). *If staging succeeds at generation g , the staged generation is $g + 1$, and*

$$g < g + 1.$$

Proof. The generation counter advances by successor in the model. The Rust implementation has additional failure cases for stale validated generations, declaration-count limits, and `u64` overflow; this theorem should not be read as a verification of those checks. $\square$

8 Lean Mechanization

The executable model is in `whitepaper/lean/IcMemory.lean`. It proves the core protocol facts over a compact abstraction of the ledger. The model avoids Rust-specific implementation details and instead captures the authority boundary that the Rust code must maintain. In particular, the Lean `OpenAuthority` type is a post-commit model object, not a field carried inside Rust’s `ValidatedAllocations` struct.

Listing 1: Core allocation records and ledger invariants.

```
abbrev StableKey := Nat
abbrev Slot := Fin 255

inductive AllocationState where
| reserved
| active
| retired

structure Record where
key : StableKey
slot : Slot
state : AllocationState
firstGeneration : Nat
lastSeenGeneration : Nat
retiredGeneration : Option Nat

def NoKeyMovement (records : List Record) : Prop :=
forall r1, r1 in records ->
forall r2, r2 in records ->
r1.key = r2.key -> r1.slot = r2.slot
```

Listing 2: The checked no-movement theorem.

```
theorem stable_key_has_unique_slot
(ledger : SafeLedger)
{r1 r2 : Record}
(h1 : r1 in ledger.records)
(h2 : r2 in ledger.records)
(sameKey : r1.key = r2.key) :
r1.slot = r2.slot :=
ledger.noKeyMovement r1 h1 r2 h2 sameKey
```

Listing 3: Post-commit opening is backed by the ledger.

```
theorem
authority_open_is_backed_by_active_ledger_record
{ledger : SafeLedger}
(authority : OpenAuthority ledger)
{key : StableKey}
{slot : Slot}
(openable : MayOpen authority key slot) :
ActiveAt ledger.records key slot := by
rcases openable with <declaration, inAuthority,
sameKey, sameSlot>
```

```
have backed := authority.sound declaration
inAuthority
...
```

The TeX listings use ASCII approximations for readability. The source file uses Lean syntax accepted by `lake build`.

9 Durable Commit Protocol

The allocation ledger is stored under the default native anchor

```
MemoryManager ID 0
→ Cell<StableCellLedgerRecord>
→ LedgerCommitStore.
```

`LedgerCommitStore` uses dual protected commit slots. Each slot contains a generation number, marker, checksum, and encoded ledger payload. Recovery chooses the highest-generation valid slot. A corrupt newer slot cannot override an older valid slot, and ambiguous equal-generation divergent slots fail closed.

The logical payload is wrapped in a small envelope before ledger decode:

```
ICMEMLD || envelopeVersion
|| ledgerSchemaVersion || physicalFormatId
|| payloadLength || CBOR(AllocationLedger).
```

This ordering matters: version routing happens before deserializing the ledger DTO. The decoded DTO still is not authority; it becomes useful for allocation authority only after compatibility checks, committed-integrity validation, and `RecoveredLedger` construction succeed.

10 Operational Guidance

Use `ic-memory` when a canister has multiple stable stores, generated stores, framework-owned stores, or plugin-provided stores that may evolve across releases. Keep stable keys stable. If the durable store is the same, preserve the key and update schema metadata; changing the key declares a new allocation identity.

The normal integration pattern is to declare ranges with `ic_memory_range!`, open through `ic_memory_key!`, and invoke `bootstrap_default_memory_manager()` from `init` and `post_upgrade` before any stable structure is touched.

Exactly one layer should bootstrap a given ledger store. Framework stacks should compose declarations into that owner, or use distinct ledger stores and allocation domains. Omitting a historical declaration does not retire or free its key or slot; explicit retirement creates a tombstone and still does not make the slot reusable for a different stable key.

11 Limitations and Non-Goals

The Lean model proves abstract allocation-ownership lemmas, and the Rust crate enforces the corresponding invariants through recovery, validation, staging, commit, and tests. This is not a formal verification of the Rust implementation. It also does not prove

semantic correctness of stored bytes, provide cryptographic tamper resistance, protect against malicious controllers, validate schema migrations, authorize endpoints, or provide disaster recovery. Its checksum protects against torn writes and accidental corruption only.

The present native slot model follows `ic-stable-structures` `MemoryManager` IDs exactly. Moving beyond 255 virtual memories would require a different slot descriptor and memory-manager layout, not merely a different `ic-memory` policy.

12 Conclusion

`ic-memory` turns stable-memory allocation from implicit convention into durable protocol state. Its central guarantee is simple: a stable key cannot move, and a physical slot cannot be silently reused. By enforcing that guarantee before memory handles are opened, the crate closes a practical upgrade hazard for multi-store Internet Computer canisters.